

# ThreatTrace AI: Intelligent Email Forensics & Threat Attribution Platform

## Daily Engineering Work Log & Progress Report — Day 16

**Sprint Phase:** Sprint Phase 5: Production Deployment & Scale | **Status:** Completed & Verified | **Schedule:** Day 16

---

### 1. Daily Sprint Overview and Context

On Day 16, the engineering team executed the final production deployment phase, transition to 24/7 cloud infrastructure, and cross-platform reliability hardening. The FastAPI backend was containerized and deployed on Render with asynchronous Gunicorn/Uvicorn workers, the PostgreSQL database was migrated to serverless Neon.tech with connection pooling and SSL enforcement, and both the primary Forensic Cockpit and the Cybercrime Officer Portal were deployed globally on Vercel Edge networks with automated health keepalive monitoring.

### 2. Daily Sprint Objectives & Scope

- 1) Deploy production FastAPI backend on Render (`https://threat-trace-ai.onrender.com`) with multi-worker scaling.
- 2) Configure serverless PostgreSQL connection pooling on Neon.tech with SSL enforcement and transaction safeguards.
- 3) Deploy React Frontend and Cybercrime Officer Portal on Vercel Edge CDN with custom Single Page Application rewrites.
- 4) Implement automated 5-minute external HTTP health check keepalives via UptimeRobot to eliminate cold start latency.
- 5) Conduct live end-to-end cloud validation: from in-Gmail extension scanning to cloud PostgreSQL evidence persistence.

### 3. Subsystem Architecture & Domain Scope

- 1) **Cloud Backend API Cluster:** FastAPI on Render with 4 async Uvicorn worker processes and CORS domain whitelisting.
- 2) **Serverless Database Tier:** Neon PostgreSQL (`threattrace\_db`) with asyncpg connection pooling and SSL encryption.
- 3) **Edge Distribution CDN:** Vercel Edge Network serving React 18 Single Page Applications with sub-50ms global TTFB.
- 4) **Automated Uptime Sentinel:** UptimeRobot continuous health check probes pinging `/api/health` every 5 minutes.
- 5) **Browser Extension Ecosystem:** Manifest V3 package connecting seamlessly to live production HTTPS endpoints.
- 6) **Cybercrime Officer Gateway:** Standalone isolated law enforcement portal deployed with officer PIN authentication.

#### **4. Individual Team Member Technical Contributions (Part I)**

The following logs detail the specific engineering contributions, algorithmic implementations, and code deliverables produced by team members during Day 16:

##### **1) Muniswami.Y — Lead AI & NLP Engineer**

- a. **Domain Ownership:** Inference Optimization & Cloud NLP Hardening
- b. **Core Technical Work:** Optimized model inference routines for the cloud production environment. Pre-warmed Bayesian prior weight caches in memory during container startup to achieve sub-120ms response times for deep NLP intent analysis. Implemented request payload sanitizers to guard against oversized email body denial-of-service attempts.
- c. **Key Tasks Executed:**
  - i. Configured startup model warmup handlers in ``backend/app/main.py``.
  - i. Conducted load testing on ``/api/analyze`` handling 50 concurrent requests with zero memory leaks.
  - i. Hardened regex evaluation routines against catastrophic backtracking (ReDoS protection).
  - i. Verified zero-log memory clearance post-inference across all cloud container instances.
- d. **Code Artifacts & Deliverables:** Model Warmup Service, ReDoS Benchmark Suite, Cloud Inference Optimization Patch.

##### **2) Jeevan reddy.G — Backend & Email Forensics Lead**

- a. **Domain Ownership:** Production Docker & Gunicorn/Uvicorn Multi-Worker Clustering
- b. **Core Technical Work:** Authored the multi-stage production ``Dockerfile`` utilizing ``python:3.11-slim`` and ``libpq-dev``. Configured Gunicorn with 4 asynchronous Uvicorn worker threads to maximize concurrent request throughput. Tuned keepalive timeouts and request buffer sizes in Render deployment manifests.
- c. **Key Tasks Executed:**
  - i. Created production ``Dockerfile`` and ``Procfile`` optimized for cloud container environments.
  - i. Configured ``render.yaml`` deployment blueprint with automated environment secret injection.
  - i. Tuned Gunicorn worker concurrency parameters to handle burst traffic during security incidents.
  - i. Verified health probe endpoint (``/api/health``) returning system load and database status.
- d. **Code Artifacts & Deliverables:** Production Dockerfile, Procfile, Render Cloud Configuration (``render.yaml``).

##### **3) Lahari.K — Threat Intelligence & Network OSINT Lead**

- a. **Domain Ownership:** DNS Resilience & External Threat Feed Caching
- b. **Core Technical Work:** Hardened external threat intelligence and DNS lookup routines for cloud network environments. Implemented resilient DNS fallback mechanisms to handle transient cloud resolver timeouts and configured persistent caching for GeoIP and ASN queries to minimize outbound network overhead.
- c. **Key Tasks Executed:**
  - i. Configured async DNS resolver with timeout clamps (2.0s max) and multi-server fallbacks (1.1.1.1, 8.8.8.8).
  - i. Established persistent LRU cache for IP geolocation records reducing external API calls by 92%.
  - i. Configured Cloudflare WAF IP reputation headers in backend request parsers.
  - i. Verified reverse DNS lookups on cloud production IP addresses.
- d. **Code Artifacts & Deliverables:** Resilient DNS Resolver Patch, GeoIP Cache Optimization, Network Timeout Safeguards.

## **5. Individual Team Member Technical Contributions (Part II)**

Detailed technical logs and code deliverables for systems architecture, graph intelligence, and frontend engineering:

### **4) Hemanth.P — Graph & Campaign Intelligence Lead**

- a. **Domain Ownership:** Edge CDN Optimization & Graph Serialization
- b. **Core Technical Work:** Optimized graph serialization algorithms to deliver large nodal attack graphs in lightweight compressed JSON payloads. Configured Vercel Single Page Application (SPA) routing rules (`vercel.json`) to prevent 404 errors on browser page reloads across deep nested routes.
- c. **Key Tasks Executed:**
  - i. Created `vercel.json` configuration with rewrite rules directing all routes to `/index.html`.
  - i. Configured gzip/brotli asset compression headers on Vercel deployment.
  - i. Optimized graph adjacency list JSON encoders to cut response payload sizes by 64%.
  - i. Benchmarked graph rendering performance on mobile and desktop edge client devices.
- d. **Code Artifacts & Deliverables:** Vercel SPA Configuration (`vercel.json`), Graph Compression Module, Edge CDN Setup.

### **5) Abbas.N — Systems Architect & Risk Engine Lead**

- a. **Domain Ownership:** Cloud Database Migration & PostgreSQL SSL Hardening
- b. **Core Technical Work:** Provisioned and hardened the serverless PostgreSQL database on Neon.tech (`threattrace\_db`). Configured SQLAlchemy async engine with `asyncpg` connection pooling, automatic connection recycling (1800s), and strict SSL certificate verification (`sslmode=require`). Applied production schema migrations.
- c. **Key Tasks Executed:**
  - i. Executed production database schema migrations on Neon Cloud PostgreSQL instance.
  - i. Configured `asyncpg` connection pool with min\_size=5, max\_size=20, and pool\_recycle=1800.
  - i. Configured UptimeRobot automated 5-minute health check monitor on live cloud backend URL.
  - i. Deployed standalone Cybercrime Officer Portal on Vercel (`threattrace-cybercrime.vercel.app`).
- d. **Code Artifacts & Deliverables:** Neon Database Setup (`neon.ts`), Async Connection Pool Layer, Uptime Sentinel Monitor.

### **6) Moulaanbee.N — Frontend & Visualization Lead**

- a. **Domain Ownership:** Cross-Browser Cloud Testing & Live UI Deployment
- b. **Core Technical Work:** Conducted comprehensive cross-browser and responsive UI testing on the live Vercel production frontend across Google Chrome, Microsoft Edge, Mozilla Firefox, and Brave. Fixed minor layout shifts, optimized SVG graph rendering performance, and verified real-time cloud API status indicators.
- c. **Key Tasks Executed:**
  - i. Deployed production frontend to Vercel (`https://threat-trace-ai.vercel.app`).
  - i. Tested live browser extension connecting to production HTTPS endpoints with zero SSL warnings.
  - i. Optimized responsive UI breakpoints for 13-inch laptop screens and high-resolution 4K displays.
  - i. Verified instant zero-login popup access and live Gmail shield button overlays.
- d. **Code Artifacts & Deliverables:** Live Vercel Production Dashboard, Extension Production Build v1.3.1, Responsive UI Patches.

## **6. System Data Flow and Module Integration**

The system data pipeline operates across sequential stages to transform raw MIME email inputs into forensic intelligence:

- 1) **Client-Side Ingestion (Browser Extension):** Content scripts in ``content.js`` observe the Gmail DOM and extract raw RFC 822 MIME headers and text bodies upon user action.
- 2) **MIME & Received Header Traversal:** The FastAPI backend traverses the ``Received`` header chain in reverse order, filtering internal relays to identify the true public originating IP address.
- 3) **Cryptographic Authentication & DNS:** The asynchronous DNS engine queries SPF TXT records, DKIM public keys, and DMARC policies to verify cryptographic signature validity.
- 4) **IOC Extraction & Network Enrichment:** URLs are defanged and deobfuscated; originating IPs are enriched with GeoIP coordinates, ASN numbers, and threat intelligence feed lookups.
- 5) **Bayesian Risk Scoring & Graph Mapping:** The composite Bayesian risk formula calculates a 0-100 score while the Graph Engine connects entities into the Threat Relationship Graph.
- 6) **Real-Time UI Dispatch:** Enriched telemetry is broadcast to the React Forensic Cockpit and Cybercrime Portal queues for security analyst review.

## **7. Technical Challenges Faced and Engineering Solutions**

### **1) Technical Challenge: Cloud Container Spin-Down Latency (Cold Starts)**

- a. **Problem Description:** Free-tier cloud hosting platforms spin down inactive containers after 15 minutes of inactivity, resulting in 45-second cold start delays for the first user.
- b. **Root Cause Analysis:** Free-tier cloud hosting platforms spin down inactive containers after 15 minutes of inactivity, resulting in 45-second cold start delays for the first user.
- c. **Engineering Solution Implemented:** Configured an automated UptimeRobot HTTP monitor that sends a lightweight keepalive ping to ``/api/health`` every 5 minutes, maintaining active container memory 24/7.

### **2) Technical Challenge: PostgreSQL SSL Handshake Drops & Pool Exhaustion**

- a. **Problem Description:** Under burst concurrent traffic, serverless PostgreSQL dropped non-SSL connections and experienced connection pool exhaustion.
- b. **Root Cause Analysis:** Under burst concurrent traffic, serverless PostgreSQL dropped non-SSL connections and experienced connection pool exhaustion.
- c. **Engineering Solution Implemented:** Configured ``asyncpg`` connection pooling with explicit ``sslmode=require``, connection recycling every 1800s, and a connection pool size of 20 with graceful fallback queues.

### **3) Technical Challenge: Single Page Application (SPA) Deep-Link 404s**

- a. **Problem Description:** Direct browser navigation or hard refreshes on nested dashboard routes (``/cockpit``, ``/cybercrime``) resulted in 404 Not Found errors on edge CDN servers.
- b. **Root Cause Analysis:** Direct browser navigation or hard refreshes on nested dashboard routes (``/cockpit``, ``/cybercrime``) resulted in 404 Not Found errors on edge CDN servers.
- c. **Engineering Solution Implemented:** Authored ``vercel.json`` routing rules and public ``_redirects`` files containing ``/* /index.html 200`` rewrites to guarantee seamless client-side routing.

---

#### 4) Technical Challenge: Cross-Origin Resource Sharing (CORS) Preflight Blocks

- a. **Problem Description:** The browser extension and independent Vercel domains were blocked by backend CORS preflight checks during cross-origin POST requests.
- b. **Root Cause Analysis:** The browser extension and independent Vercel domains were blocked by backend CORS preflight checks during cross-origin POST requests.
- c. **Engineering Solution Implemented:** Configured explicit FastAPI `CORSMiddleware` with allowed origins covering all deployment domains (`https://threat-trace-ai.vercel.app`, `https://threattrace-cybercrime.vercel.app`, and `chrome-extension://\*`).

## **8. Quality Assurance, Test Suite Metrics & Verification**

Automated test suites were executed against Day 16 builds to verify RFC compliance and sub-200ms processing latencies:

- 1) **Test Case TC-16-01 (Live Cloud Backend Health Check):** Input: *GET https://threat-trace-ai.onrender.com/api/health* → Expected: *HTTP 200 with DB Status: CONNECTED* → **Status: PASS**
- 2) **Test Case TC-16-02 (Cloud PostgreSQL Transaction Test):** Input: *Async Case Insert & Query via Neon DB* → Expected: *Sub-15ms Read/Write with Zero Connection Drop* → **Status: PASS**
- 3) **Test Case TC-16-03 (Vercel Edge CDN Deployment):** Input: *GET https://threat-trace-ai.vercel.app* → Expected: *HTTP 200 with Compressed Assets (<50ms TTFB)* → **Status: PASS**
- 4) **Test Case TC-16-04 (Extension Cloud Integration):** Input: *Live Gmail Scan & Dispatch to Cloud API* → Expected: *100% End-to-End Threat Scoring in Live UI* → **Status: PASS**
- 5) **Test Case TC-16-05 (24/7 Uptime Sentinel Probe):** Input: *UptimeRobot 5-Min HTTP Keepalive Ping* → Expected: *100% Uptime Across 24-Hour Continuous Window* → **Status: PASS**

## **9. Security, Data Privacy and Chain-of-Custody Compliance**

- 1) **Zero-Log Ephemeral Processing:** Raw email content is analyzed in volatile memory and is never permanently written to disk without explicit user reporting.
- 2) **Cryptographic Evidence Hashing:** All extracted email headers, body texts, and attachment metadata are hashed using canonical SHA-256 for court-admissible chain of custody.
- 3) **SSRF Protection:** Network URL deobfuscators strictly block loopback, private RFC 1918 subnets, and cloud metadata IP addresses (169.254.169.254).

## **10. Daily Milestone Verification & Next Day Schedule**

- 1) **Day 16 Milestone Status:** 100% of scheduled tasks, API endpoints, and test suites completed and verified.
- 2) **Planned Objectives for Day 16:** Post-deployment monitoring, store certification maintenance, and continuous threat intelligence feed ingestion.
- 3) **Repository Synchronization:** All tested code committed and pushed to main Git repository.